

Raku++

A from-scratch Raku, hand-written in C++17.

Interpreter + native compiler

runs in the **browser** via WebAssembly

Zero third-party dependencies

v1.0.0

— NOT A RAKUDO FORK — SHARES NO CODE WITH IT

One implementation of the **language**, from lexer to native binary.

A hand-written lexer, parser, and tree-walking evaluator for real **Raku** — classes, roles, grammars, regexes, multi-dispatch, junctions, lazy sequences, a bignum tower, Unicode-correct strings, concurrency. It can also **compile** a program to a standalone native executable, and — as **Raku.js** — run client-side in a browser.

- **C++17, no deps**

Hand-rolled bignum, Unicode tables, and regex engine. Builds anywhere CMake and a C++17 compiler run.

- **Interpret or compile**

The same binary tree-walks a script or transpiles it to C++ and compiles native.

- **Measured, not claimed**

Graded test-by-test against Roast, the official Raku specification suite.

— GRADED AGAINST ROAST · V1.0.0'S 90% TARGET, REACHED

How much Raku actually runs.

90%

194,506 / 216,066 declared tests pass —
97.4% of the tests that actually run.

584 / 1,462

files pass on the strict **all-or-nothing** bar —
every assertion in the file must pass (~39%).

~3 $\frac{1}{2}$ min

to run the whole suite through a **self-hosted harness** — written in Raku, run by Raku++ itself.

Every release is gated on the full suite with **zero fully-passing-file regressions**.
Numbers are measured on each tag, never projected.

— THE BREADTH OF WHAT'S IMPLEMENTED

Big-language Raku, most corners lit.

NUMBERS

Exact by default

Arbitrary-precision `Int`,
exact `Rat` — `0.1+0.2-0.3`
is exactly `0` — `Num`,
`Complex`, allomorphs.

OBJECTS

class · role · grammar

Multi-dispatch,
`BUILD` / `TWEAK`, mixins,
`augment`, full `.^`
introspection, `.wrap`.

REGEX

Grammars that parse

Backtracking engine, Unicode
classes, `token` / `rule`,
`.parse` with action classes &
`make`.

SIGNATURES

multi & proto

Type / literal / `where`
constraints, destructuring,
coercion types, `callsame`.

FUNCTIONAL

Lazy & higher-order

Closures, `*` `WhateverCode`,
junctions, `gather` / `take`,
feeds, `1,2,*+*...Inf`.

METAPROG

Your own operators

User ops in all six categories
with working precedence
traits and meta-operators over
them.

CONCURRENCY

Real OS threads

`start` / `await`,
`Supply` / `react` / `whenever`,
`Channel`, atomics; opt-in
true parallelism.

UNICODE

Grapheme-correct

UAX #29 (emoji ZWJ),
NFC/NFD/NFKC/NFKD, UCA
collation — generated
UCD/UCA 17.0 tables.

— ONE SOURCE, FIVE TARGETS

Four ways to run — and a fifth in the browser.

rakupp prog.raku

Interpret — the default; covers the whole language.

--bundle -o prog

Standalone binary that re-parses the source at startup.

--aot -o prog

Standalone binary embedding the already-parsed AST.

--exe -o prog

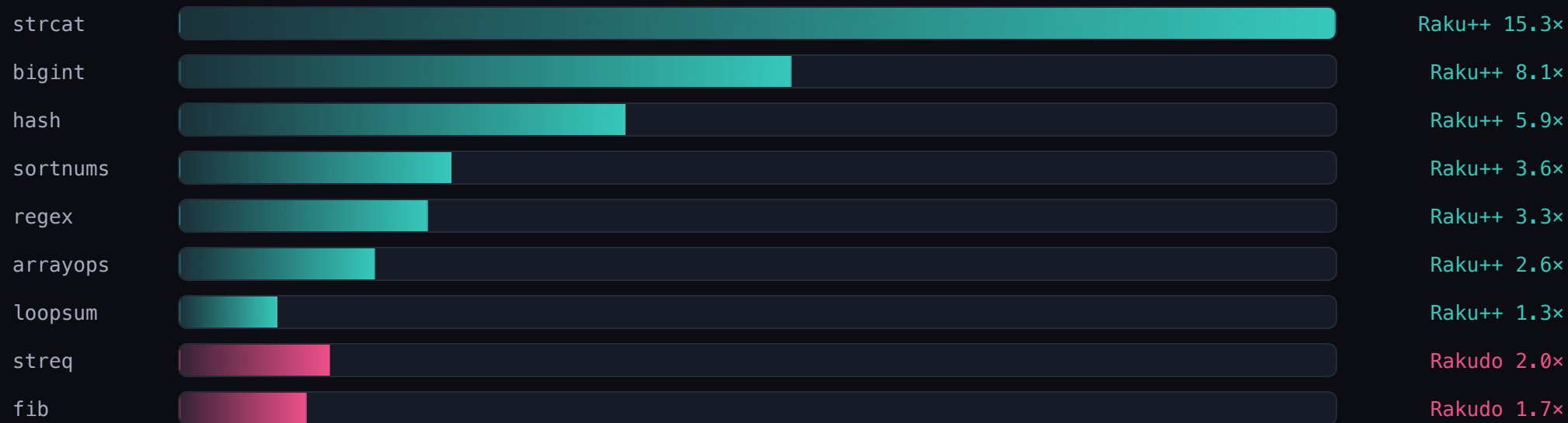
Transpile to C++ and compile native — anything unsupported falls back to **--bundle** automatically.

Raku.js → WASM

In the browser — same semantics as native, client-side, no server, with an embeddable playground.

— INTERPRETER · × VS RAKUDO V2026.06 · BYTE-IDENTICAL OUTPUT GATE

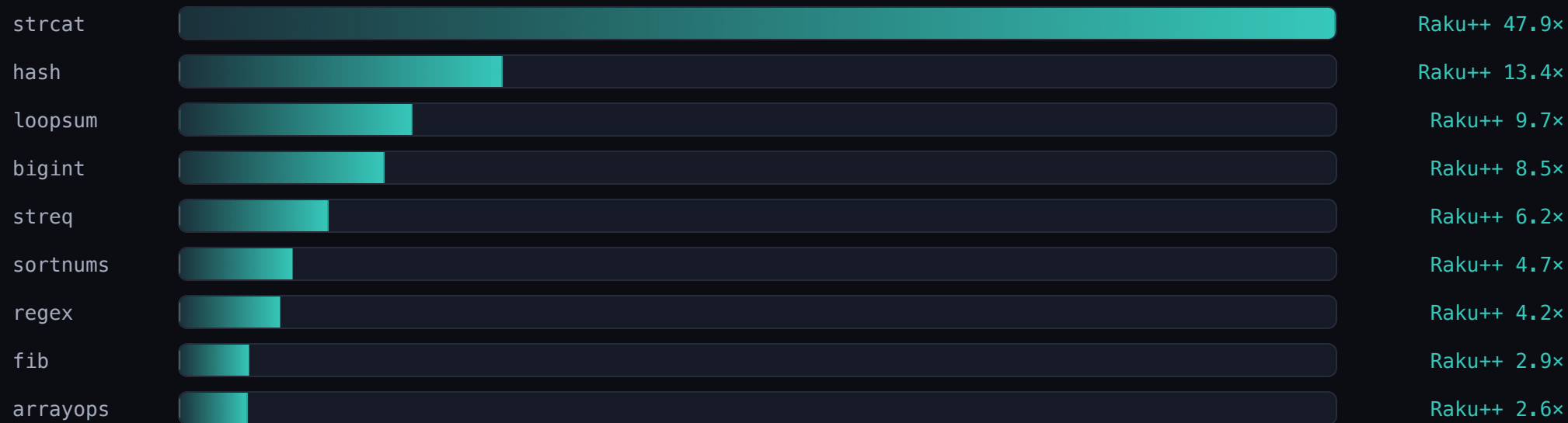
Fast to start, fast to run.



~2 ms cold start (best 1.8 ms of a 200-spawn loop) – a tiny native binary, no VM to spin up. Even [interpreted](#), Raku++ wins 7 of 9 kernels; the two it trails on – streq, fib – the next slide compiles away.

— RAKUPP --EXE · TRANSPILED TO C++, COMPILED NATIVE · × VS RAKUDO

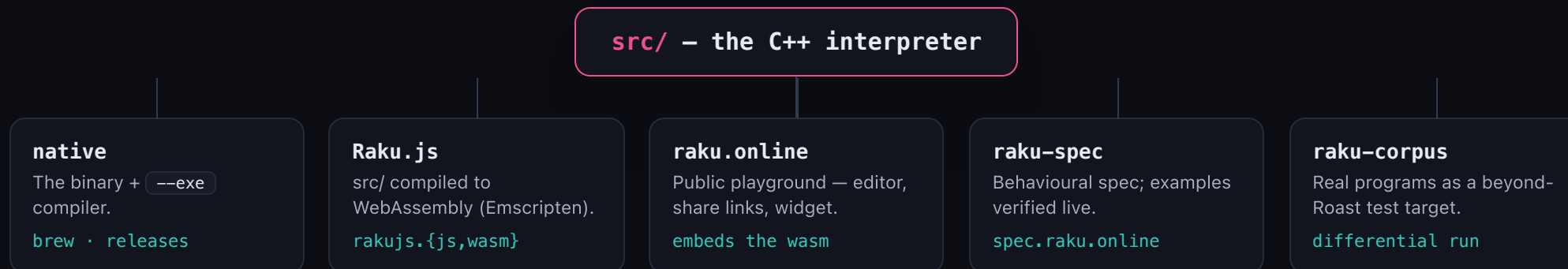
Compiled, it wins **every** kernel.



--exe strips the tree-walk: every benchmark lands 2.6× to 47.9× ahead of Rakudo – streq and fib, the interpreter's only two losses, now included. Anything the compiler can't lower falls back to bundling automatically.

— ONE INTERPRETER BEHIND EVERY SURFACE

A small constellation, one source of truth.



Everything downstream ships the *same* interpreter — the `wasm`, the two sites, the Homebrew tap, and the release binaries all trace back to `src/`. The spec inherits a new engine for free the moment the playground redeploys.

— MID-SIZE PROGRAMS, EACH LEANING ON A DIFFERENT PART OF THE LANGUAGE

Real programs, written **in** Raku.

Scheme

A Lisp interpreter on a Raku grammar.

GRAMMAR · EVAL

Forth

Stack machine, also grammar-parsed.

GRAMMAR

Markdown → HTML

Converter — also runs in the browser.

REGEX · WASM

JSON

Parser + formatter, browser-ready too.

GRAMMAR · WASM

Pastebin

HTTP service on raw sockets.

SOCKETS

Chat server

Concurrent, many-client.

THREADS

Key-value store

Its own wire protocol.

SOCKETS

rakus

A static-file HTTP server.

SOCKETS

Plus larger experiments — a **JavaScript/TypeScript** interpreter, a **Perl 5** interpreter (Raku parsing its own ancestor), and a **raytracer**. Every one of the 24 bundled `examples/` also compiles natively with `--exe`, byte-for-byte identical.

— BEYOND WHAT ROAST'S UNIT GRANULARITY EXERCISES

Real applications run **unmodified**.

RAKU-COURSE

~1,500 pages, byte-identical

The full course static-site generator regenerates the entire site byte-for-byte identically to Rakudo — including the real zef-installed

`YAMLish` module.

COVID.OBSERVER

End-to-end build

The site builder runs start to finish on Raku++ — a genuine data-processing application, not a curated snippet.

```
# exact arithmetic, lazy infinite sequence, grapheme strings –
# the things that make real programs match, not just tests
say 0.1 + 0.2 - 0.3;           # 0 (exact Rat)
say (1, 2, *+* ... *)[^10];   # 1 1 2 3 5 8 13 21 34 55
say "café".chars;              # 4 (graphemes, not bytes)
```

— HONEST ABOUT THE EDGES

Not there yet.

- **Macros / RakuAST / slangs** — compile-time metaprogramming and pluggable syntax.
- **libffi-grade NativeCall** — structs and callbacks (basic C FFI to libc / `<math.h>` already works, without libffi).
- Some **IO** and **POD** corners.
- **Lock-free parallel atomics.**

Early-stage and growing test-first — the direction is more of Roast, passing file by file, with every release gated on no regressions.

— MACOS · LINUX · WINDOWS · THE BROWSER

Try **Raku++** today.

```
$ brew tap ash/rakupp # macOS
```

```
$ brew install rakupp
```

```
➤ raku.online # run it in the browser, no install
```

github.com/ash/rakupp

raku.online

spec.raku.online

Prebuilt archives on every GitHub Release — macOS universal, Linux static, Windows static-CRT — plus the WebAssembly bundle. No runtime dependencies, anywhere.